

Preprocessing Pipeline

Hospital Readmission Risk Prediction

Dataset: Hospital Readmission Risk Prediction

Records: 50,500 raw → 50,000 after cleaning

Features: 28 raw → 31 final modeling features

Target: readmitted (0 = Not Readmitted, 1 = Readmitted)

Documented By: Nithish G

AI/ML Engineer

Step 1: Duplicate Removal

Problem: 500 exact duplicate rows present in the raw dataset.

Observations:

- 500 rows were exact copies across all 28 columns.
- Duplicate patient_id count matched duplicate row count exactly (500).
- Dataset reduced from 50,500 → 50,000 rows after removal.

Decision: Drop all exact duplicate rows using drop_duplicates(); reset index.

Reason:

- Identical rows provide zero additional information to the model.
- Retaining duplicates inflates sample weight for specific records and skews frequency statistics.
- The duplication pattern (exactly 500 rows) points to a double-write in the data pipeline, not real repeat admissions.

Alternatives Considered & Rejected:

- *Keep duplicates* → *Rejected: Creates artificial over-representation of those records.*
- *Deduplicate by patient_id only* → *Rejected: 500 IDs matched 500 full rows; result is identical but hides whether values differed.*

Step 2: Invalid Values

Problem: Several numeric columns contained physiologically impossible values.

Observations:

- patient_age: ~202 entries outside the valid 0–120 range.
- heart_rate: ~101 entries above 250 bpm — no living patient sustains this.
- bmi: entries below 10 are clinically implausible for a hospital patient.
- These are data entry errors, not statistical outliers.

Decision: Set out-of-range values to NaN before imputation for all three columns.

Reason:

- Nulling invalid values preserves all other column data in those rows.

- Thresholds are set on physiological bounds, not percentile cutoffs — avoids removing valid extreme values.
- Separating nullification from imputation keeps each step's purpose clear.

Alternatives Considered & Rejected:

- *Clip to valid range → Rejected: Keeps the extreme value rather than treating it as truly missing.*
- *Row deletion → Rejected: Discards 25+ valid data points per row over one bad column entry.*

Step 3: Categorical Inconsistency — gender

Problem: Same categories represented in multiple formats.

Observations:

- gender had 8 variants: Male, M, male, MALE, Female, F, female, FEMALE.
- Direct encoding without cleaning creates phantom categories treated as separate classes.
- Every other categorical column was already consistent — gender was the only field missing input validation at the source system.

Decision: Map all variants to canonical title-case values: 'Male' / 'Female'.

Reason:

- A single `.map()` dictionary call handles all case and abbreviation variants in one pass.
- Post-mapping the column is a clean binary field with no unmapped entries.

Alternatives Considered & Rejected:

- *`str.title()` directly → Rejected: Cannot handle abbreviations like 'M' or 'F'.*
- *Label encode without normalization → Rejected: Would encode 'male' and 'Male' as separate numeric classes.*

Step 4: Type Conversion — hospital_bill_amount (₹ Symbol)

Problem: hospital_bill_amount stored as object dtype due to embedded ₹ symbol.

Observations:

- Values like '₹125345' cannot be read as numeric directly.
- `pd.to_numeric()` rejects non-numeric characters regardless of context.
- `errors='coerce'` needed to handle edge cases like '₹ N/A' without crashing.

Decision: Strip ₹ symbol as a string operation first, then convert with `pd.to_numeric(errors='coerce')`.

Reason:

- String stripping handles all variants including '₹ ' (with trailing space).
- `errors='coerce'` converts remaining unparseable values to NaN for imputation rather than raising exceptions.

Alternatives Considered & Rejected:

- *`astype(float)` directly → Rejected: Throws `ValueError` on symbol-containing strings.*
- *Regex extraction → Rejected: Adds unnecessary complexity for a simple prefix strip.*

Step 5: Date Format Standardisation

Problem: Date columns contained mixed formats across rows.

Observations:

- Formats present: YYYY-MM-DD and MM-DD-YYYY (at minimum in followup_date).
- A single fixed format string silently misparsed or errored on other convention rows.
- All three date columns (admission_date, discharge_date, followup_date) needed parsing before feature extraction.

Decision: Parse with `pd.to_datetime(format='mixed', errors='coerce')` for all three date columns.

Reason:

- `format='mixed'` handles multiple observed date conventions without manual detection logic.
- `errors='coerce'` converts unparseable entries to NaT without stopping the pipeline.
- Proper datetime objects enable direct subtraction for `length_of_stay` and `followup_delay`.

Alternatives Considered & Rejected:

- *Manual format detection per row → Rejected: Fragile and verbose for multiple formats.*
- *Single fixed format string → Rejected: Silently misparses rows in a different convention.*

Step 6: Missing Value Imputation

Problem: Multiple columns contained missing values after type conversion and invalid-value nulling.

Observations:

- `patient_age`: ~202 missing (~0.4%) after physiological nulling.
- `glucose_level`: 7,505 missing (15.01%).
- `doctor_rating`: 7,505 missing (15.01%).
- `followup_compliance_score`: 7,470 missing (14.94%).
- `hospital_bill_amount`: 6,435 missing (12.87%) — skewness ~30.49, mean heavily distorted by extreme high bills.
- `heart_rate`: ~101 missing (~0.2%) after physiological nulling.
- `bmi`: ~101 missing (~0.2%) after physiological nulling.
- `secondary_diagnosis`: 25.18% missing — semantically means 'no secondary condition', not unknown.

Decision: Numeric columns → Median imputation. `secondary_diagnosis` → Fill with 'None' (sentinel string).

Reason:

- Median is robust to extreme values — unaffected by `hospital_bill_amount`'s ₹1 crore outlier.
- Treating `secondary_diagnosis` NaN as 'None' is semantically correct: a missing secondary diagnosis means no secondary condition was recorded.
- Result: 0 missing values across all numeric columns after this step.

Alternatives Considered & Rejected:

- *Mean imputation → Rejected: Mean distorted by outliers in hospital_bill_amount (skewness 30.49).*
- *KNN imputation → Rejected: Computationally expensive on 50,000 rows; negligible lift given near-symmetric distributions.*
- *Forward/backward fill → Rejected: Assumes temporal ordering which this dataset does not have.*

Step 7: Feature Engineering

Problem: Raw columns do not directly expose the clinical and behavioural patterns most associated with readmission risk.

Observations:

- Negative length_of_stay values (~3,000 rows) indicated swapped admission/discharge dates.
- followup_delay had negative values where followup_date was logged before discharge_date.
- Admission season shows a distinct Monsoon spike (40.4%) — disease seasonality is a real predictor.
- previous_admissions + previous_emergency_visits are uncorrelated with each other but both predict readmission.
- secondary_diagnosis presence vs. absence is more actionable than the raw text value.
- blood_pressure as a continuous value is harder to interpret than a clinical category.
- followup_compliance_score < 50 flags patients with weak post-discharge engagement.

Decision: Create 8 engineered features:

- length_of_stay: (discharge_date – admission_date).days — swap dates in-place where negative.
- followup_delay: (followup_date – discharge_date).days.
- admission_month / admission_year: extracted from admission_date.
- admission_season: Winter / Summer / Monsoon / Post-Monsoon based on admission month.
- total_risk_history: previous_admissions + previous_emergency_visits.
- has_secondary_diagnosis: binary flag (1 = secondary condition present).
- bp_category: Low / Normal / Elevated / High based on blood_pressure thresholds.
- medication_intensity: medication_count / (lab_test_count + 1) — volume-independent treatment ratio.
- low_compliance: binary flag for followup_compliance_score < 50.

Reason:

- Swapping dates in-place for negative stays preserves the actual duration; nulling would permanently discard valid clinical data.
- total_risk_history reduces two inputs to a single prior-burden signal without losing information from either.
- admission_season encodes real-world disease seasonality — Monsoon drives waterborne and respiratory illness spikes in Indian healthcare.
- bp_category converts a near-uniform continuous distribution into a clinically interpretable ordinal feature.
- medication_intensity normalization makes the metric volume-independent across all patient scales.
- low_compliance threshold of 50 on a 0–100 scale treats 'below half' as clinically meaningful non-adherence.

Alternatives Considered & Rejected:

- *Raw counts without normalization → Rejected: High-utilization patients appear riskier purely due to larger absolute counts.*
- *Finer thresholds → Rejected: Would require holdout-set validation; chosen thresholds are operationally interpretable without overfitting.*
- *Keep raw date columns only → Rejected: Direct subtraction features carry more predictive information than raw timestamps.*

Step 8: Data Leakage Removal

Problem: Three columns contain information that is unavailable at the time a readmission risk assessment is made.

Observations:

- readmission_notice_sent: sent only after a readmission event occurs — does not exist at prediction time.
- readmission_confirmed: IS the event we are predicting — including this is direct label leakage.
- final_readmission_reason: known only after readmission — future information.
- These columns showed low direct correlation with readmitted — but leakage is about TIMING, not correlation.

Decision: Drop: readmission_notice_sent, readmission_confirmed, final_readmission_reason.

Reason:

- A model trained on post-event columns would score perfectly in training but produce zero value in production.
- All remaining features are derivable from data present at the point of discharge or admission.

Alternatives Considered & Rejected:

- *Keep low-correlation leakage columns → Rejected: Correlation is irrelevant; timing is the only valid leakage criterion.*
- *Use readmission_confirmed with domain rules → Rejected: Column never available before the system makes its initial prediction.*

Step 9: Removing Identifier and Raw Date Columns

Problem: Identifier and raw date columns add no predictive signal and can cause overfitting.

Observations:

- patient_id: a unique identifier per row — encodes no generalisable pattern.
- admission_date, discharge_date, followup_date: all temporal information has been captured in engineered features.

Decision: Drop patient_id and the three raw date columns.

Reason:

- Identifiers can cause the model to memorise individual patients rather than learning generalisable patterns.
- Raw timestamps are redundant once length_of_stay, followup_delay, admission_month, admission_year, and admission_season are created.
- Final processed dataset: 50,000 rows × 31 feature columns + 1 target column.

Step 10: Feature Encoding

Problem: Models cannot process string categorical values directly.

Observations:

- Categorical columns: gender, marital_status, admission_type, department, diagnosis, secondary_diagnosis, insurance_type, admission_season, bp_category.
- All are nominal with no inherent ordering — ordinal encoding would imply false rank relationships.

- 4 of the 9 columns have high cardinality (diagnosis: 5 values, department: 5 values).
- Encoded training shape: 40,000 × 46 features.

Decision: One-hot encode all categorical columns using `pd.get_dummies(drop_first=True)`. Align test set columns to training set using `.align(join='left', fill_value=0)`.

Reason:

- `drop_first=True` eliminates perfect multicollinearity (dummy variable trap) in linear models.
- Test-set alignment with `fill_value=0` handles any unseen categories without runtime errors.
- One-hot is the correct choice for purely nominal columns — no ordering exists to exploit.

Alternatives Considered & Rejected:

- *Label encoding for nominal columns → Rejected: Implies false ordinal relationships (e.g., Orthopedics > Neurology).*
- *Target encoding → Rejected: Requires careful cross-validation to avoid leakage; unnecessary given low cardinality.*

Step 11: Train-Test Split and Feature Scaling

Problem: Model needs unseen data to evaluate real-world performance; scale differences affect gradient-based models.

Observations:

- Dataset: 50,000 rows, 93.6% readmitted — stratification required to preserve class ratio.
- `hospital_bill_amount` ranges 0 to ~₹1 Cr; `patient_age` ranges 0–120 — very different scales.
- Tree-based models (Random Forest, XGBoost) do not require feature scaling.
- Class imbalance ratio: approximately 14.6:1 (readmitted vs not readmitted).

Decision: 80/20 stratified split: 40,000 train / 10,000 test. `StandardScaler` fitted on training data only. `class_weight='balanced'` and `scale_pos_weight=0.069` used in models to handle imbalance.

Reason:

- Stratification preserves the 93.6:6.4 class ratio in both train and test sets.
- Scaler fitted only on training data — fitting on the full dataset leaks test set statistics into training.
- SMOTE oversampling was also tested (37,426 synthetic minority samples) but produced negligible gain over `scale_pos_weight` alone.

Alternatives Considered & Rejected:

- *70/30 split → Rejected: 80/20 with 50,000 rows gives a reliable 10,000-row test set.*
- *Min-Max scaling → Rejected: Sensitive to outliers; StandardScaler more robust given wide feature ranges.*
- *SMOTE only → Rejected: Increases computation with minimal performance gain over class weighting (recall delta +0.001).*
- *No scaling → Acceptable for tree models; StandardScaler applied once and used across all model types for consistency.*